

Week02 实验报告

mrw1114

August, 31st

1 个人信息

- 姓名：王俊杰
- 学号：25020007122
- Github 仓库链接：<https://github.com/mrw1114/fundamentals-of-system-dev-tools>

2 课上实验

2.1 第 05 题：控制一个可清理的后台任务

2.1.1 保存脚本，赋予权限，并在前台启动

将内容保存到 `worker.sh`，用 `chmod` 赋予可执行权限，启动脚本并把输出流和错误流分别重定向到 `stdout.log` 和 `stderr.log`。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ vim ./worker.sh
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ cat ./worker.sh
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ chmod +x ./worker.sh
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ ./worker.sh > stdout.log 2> stderr.log

```

图 1: 运行与重定向

2.1.2 挂起程序并转到后台运行

使用 `Ctrl + Z` 将正在运行的进程挂起，再使用 `bg` 将其置于后台运行，用 `jobs` 确认其状态。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ jobs
[1]+  运行中                  ./worker.sh > stdout.log 2> stderr.log &
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$

```

图 2: 挂起与后台运行

2.1.3 获取 pid 并向其发送 SIGTERM

用 `jobs -p %1` 获取我们放入后台运行的程序的 `pid`，利用命令替换特性将其 `pid` 传入 `kill -TREM` 中从而向该进程发送 `SIGTERM`。最后检查 `cleanup.log`。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ jobs -p %1
12658
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ kill -TERM $(jobs -p %1)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ ls
cleanup.log  stderr.log  stdout.log  worker.sh
[1]+  已完成                  ./worker.sh > stdout.log 2> stderr.log
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ cat cleanup.log
CLEAN_EXIT
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$ jobs
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q05$
```

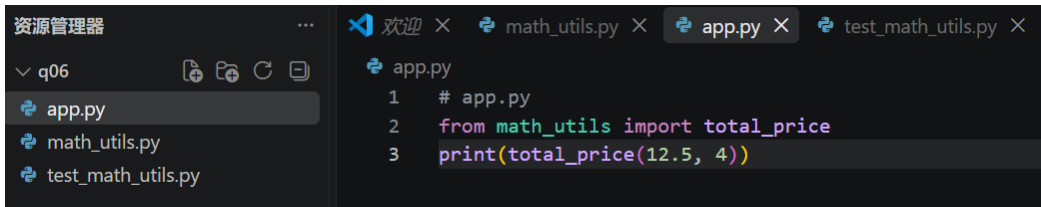
图 3: 运行与重定向

2.2 第 06 题：语义重构与本地开发反馈

由于配置问题，本题目在 Windows 环境下使用 VS Code IDE 与 Python 插件、Pylance 语言服务器进行。

2.2.1 创建文件

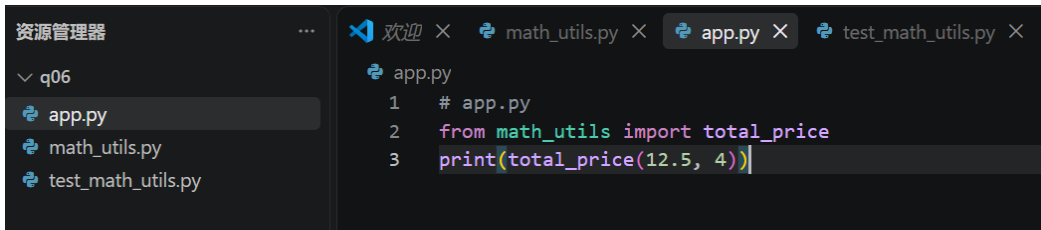
新建 q06 文件夹，用 VS Code 打开并新建对应的三个文件。



```
资源管理器
└─ q06
   ├── app.py
   ├── math_utils.py
   └── test_math_utils.py

app.py
1 # app.py
2 from math_utils import total_price
3 print(total_price(12.5, 4))
```

图 4: math_utils.py 内容



```
资源管理器
└─ q06
   ├── app.py
   ├── math_utils.py
   └── test_math_utils.py

app.py
1 # app.py
2 from math_utils import total_price
3 print(total_price(12.5, 4))
```

图 5: app.py 内容

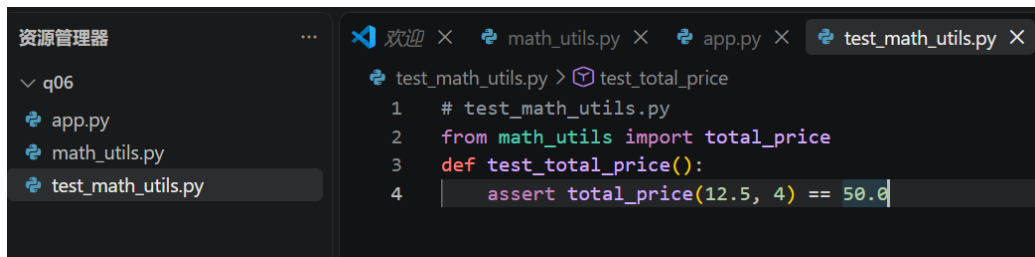


图 6: test_math_utils.py 内容

2.2.2 配置语言服务器并对 total_price 函数转到定义和查找所有引用

打开 VS Code 的设置，修改 Python Language Server 选项为 Pylance。随后打开 app.py，Ctrl + Click 即可转到其在 math_utils.py 中的定义，右键菜单可点击查找所有引用显示其引用。

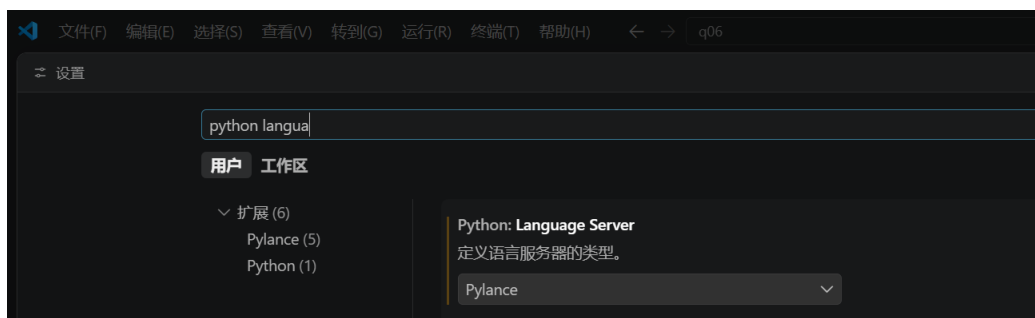


图 7: 配置语言服务器

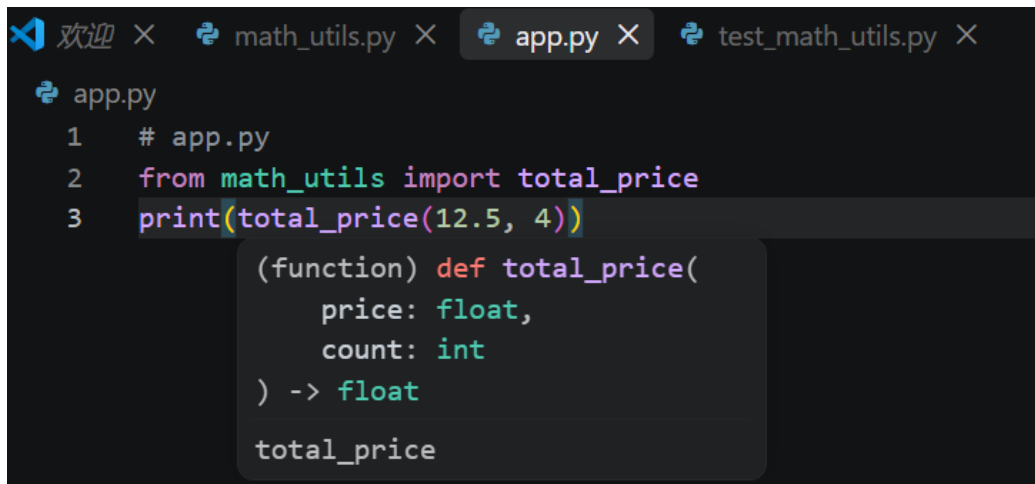


图 8: 转到 `total_price` 定义

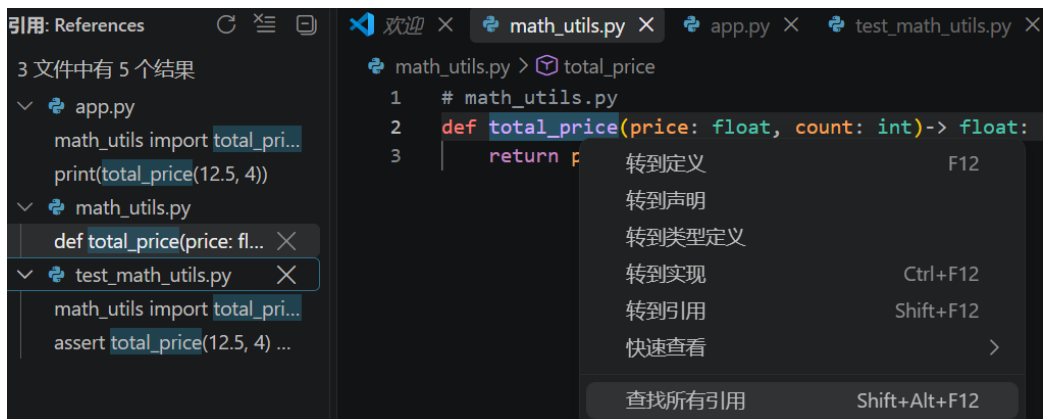


图 9: 查找 `total_price` 所有引用

2.2.3 重命名符号

右键 `total_price` 并选择重命名符号，将其命名为 `calculate_total`。

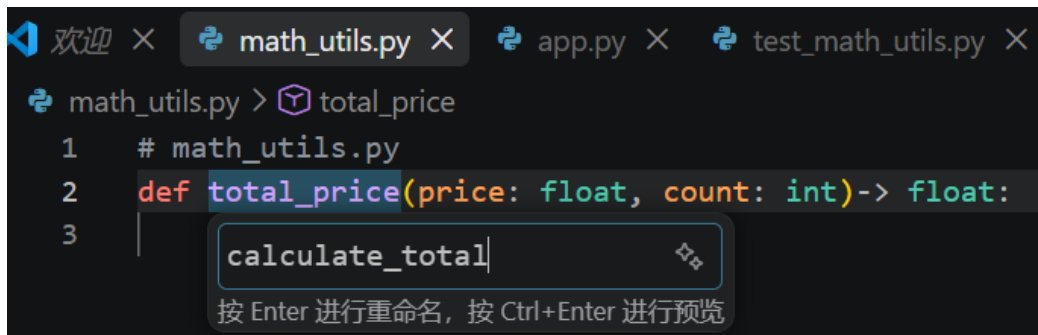


图 10: 重命名符号

2.2.4 创建文件

向 app.py 中添加未使用的 import os, 随后终端运行 ruff check . 检查当前目录下的问题, 发现 2 处格式问题和 1 处未使用的 import 。通过 ruff check . -fix 将三处问题修复。随后 pytest 检查通过

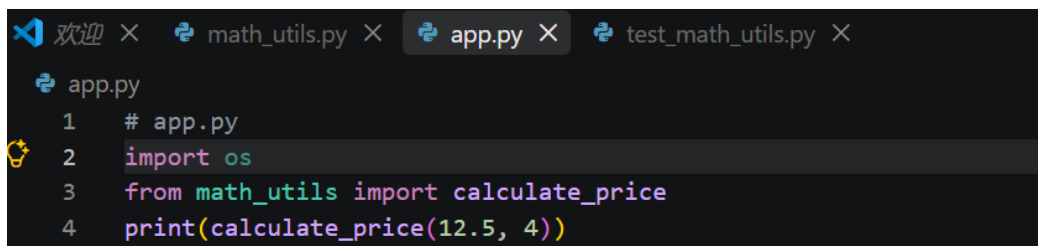


图 11: 写入未使用的 import os

```
PS C:\Users\27111\Documents\q06> ruff check .
```

```
I001 [*] Import block is un-sorted or un-formatted
```

```
--> app.py:2:1
```

```
|
1 | # app.py
2 | / import os
3 | | from math_utils import calculate_price
  | | _____^
4 | | print(calculate_price(12.5, 4))
```

```
help: Organize imports
```

```
|
2 | import os
3 | +
4 | from math_utils import calculate_price
5 | +
6 | print(calculate_price(12.5, 4))
|
```

```
F401 [*] `os` imported but unused
```

```
--> app.py:2:8
```

```
|
1 | # app.py
2 | import os
  |      ^^
3 | from math_utils import calculate_price
4 | print(calculate_price(12.5, 4))
|
```

```
help: Remove unused import: `os`
```

```
|
1 | # app.py
  | - import os
2 | from math_utils import calculate_price
|
```

```
I001 [*] Import block is un-sorted or un-formatted
```

```
--> test_math_utils.py:2:1
```

```
|
1 | # test_math_utils.py
2 | from math_utils import calculate_price
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

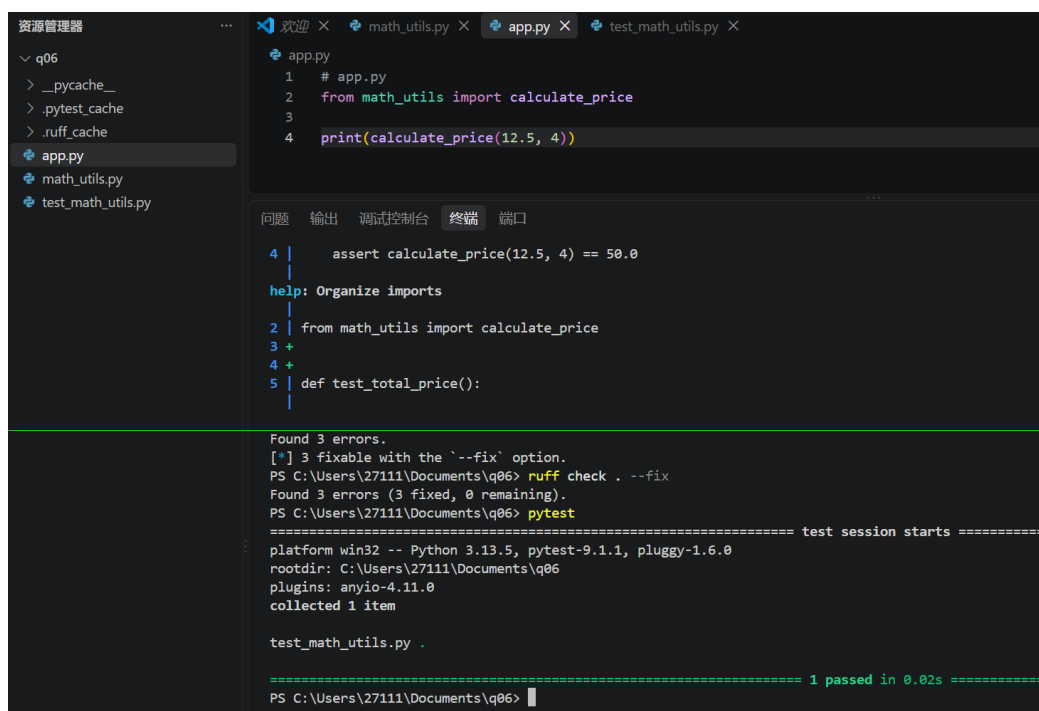


图 13: ruff 修复与 pytest 结果

2.3 第 07 题：用调试器定位归并排序缺陷

建立虚拟环境如下，安装 ruff 和 pytest，随后启用虚拟环境。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q06$ cd ../
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ python3 -m venv ./for_dbg
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ source ./for_dbg/bin/activate
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ pip install ruff pytest pyright
Collecting ruff
  Downloading ruff-0.16.5-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (26 kB)
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting pyright
  Downloading pyright-1.1.411-py3-none-any.whl.metadata (6.9 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Collecting nodeenv>=1.6.0 (from pyright)
  Downloading nodeenv-1.10.0-py2.py3-none-any.whl.metadata (24 kB)
Collecting typing-extensions>=4.1 (from pyright)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Downloading ruff-0.16.5-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (10.3 MB)
  10.3/10.3 MB 12.2 MB/s eta 0:00:00
Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
  386.5/386.5 kB 84.6 MB/s eta 0:00:00
Downloading pyright-1.1.411-py3-none-any.whl (6.2 MB)
  6.2/6.2 MB 4.3 MB/s eta 0:00:00
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading nodeenv-1.10.0-py2.py3-none-any.whl (23 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
  130.0/130.0 kB 30.0 MB/s eta 0:00:00
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
  1.3/1.3 MB 13.1 MB/s eta 0:00:00
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
  45.6/45.6 kB 13.3 MB/s eta 0:00:00
Installing collected packages: typing-extensions, ruff, pygments, pluggy, packaging, nodeenv, iniconfig, pytest, pyright
Successfully installed iniconfig-2.3.0 nodeenv-1.10.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pyright-1.1.411 pytest-9.1.1 ruff-0.16.5 typing-extensions-4.16.0
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$

```

图 14: 配置环境

2.3.1 创建有问题的归并排序代码文件并观察其输出

将有缺陷的代码写入 `merge_sort.py` 中，运行查看其输出结果。

```

(for_dbg) nrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ vim ./merge_sort.py
(for_dbg) nrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ cat ./merge_sort.py
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
(for_dbg) nrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ python3 ./merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
(for_dbg) nrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$

```

图 15: 有缺陷的代码及其输出结果

2.3.2 用 pdb 进行调试并观察 merge 执行过程中的变量变化

使用 `python -m pdb ./merge_sort.py` 进入调试器，设置断点以及中断时执行的命令，查看运行过程中 `i`, `j`, `left[i]`, `right[j]` 的变化。

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ python -m pdb ./merge_sort.py
> /home/mrw/sys-dev-test/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b 5
Breakpoint 1 at /home/mrw/sys-dev-test/q07/merge_sort.py:5
(Pdb) commands 1
(com) p i, j, left[i], right[j]
(com) c
(Pdb) c
(0, 0, 3, 1)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 0, 4, 1)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 0, 1, 1)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(1, 0, 3, 1)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(1, 1, 3, 4)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 0, 5, 9)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 0, 2, 6)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 0, 5, 2)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 1, 5, 6)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(1, 1, 9, 6)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(0, 0, 1, 2)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(1, 0, 4, 2)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(1, 1, 4, 5)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(2, 1, 3, 5)
> /home/mrw/sys-dev-test/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(3, 1, 4, 5)

```

图 16: 调试结果

2.3.3 修正问题并查看输出结果

代码中当 $\text{left}[i] > \text{right}[j]$ 时应当将 $\text{right}[j]$ 置入 result 中, 而非 $\text{right}[i]$ 。
修正代码后查看输出结果

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ vim ./merge_sort.py
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ cat ./merge_sort.py
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 确保添加的是 right[j]
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ python3 ./merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]

```

图 17: 问题修正后的输出

2.3.4 增加测试文件并使用 pytest 检验

新建 test_merge_sort.py, 从 merge_sort 函数引入 merge_sort 函数, 并定义带有断言的两个测试函数。随后 pytest . 对代码进行测试。

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ vim ./test_merge_sort.py
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ cat ./test_merge_sort.py
from merge_sort import merge_sort

# 测试1
def test1():
    input_data = [1, 1, 4, 5, 1, 4]
    result = merge_sort(input_data)
    # 断言期望输出
    assert result == [1, 1, 1, 4, 4, 5]

# 测试2
def test2():
    input_data = [1, 9, 1, 9, 8, 1, 0]
    result = merge_sort(input_data)
    assert result == [0, 1, 1, 1, 8, 9, 9]
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q07$ pytest .
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/mrw/sys-dev-test/q07
collected 2 items

test_merge_sort.py .. [100%]

===== 2 passed in 0.01s =====

```

图 18: 测试结果

2.4 第 08 题：先测量，再优化慢速词频程序

2.4.1 新建对应文件

将对应代码输入对应文件中。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ vim ./generate_words.py
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ vim ./wordfreq.py
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ cat ./generate_words.py ./wordfreq.py
# generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$
```

图 19: 新建相关文件

2.4.2 生成词库，运行原始词频统计程序并记录运行时间

运行 generate_words.py 生成词库 words.txt，随后使用 time 命令运行 ./wordfreq.py 程序两次并记录运行时间。得到一开始的程序运行时间分别为 0.112s 和 0.117s。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ python3 ./generate_words.py
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ ls -l ./words.txt
-rw-rw-r-- 1 mrw mrw 146755 Aug 31 15:26 ./words.txt
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ time python wordfreq.py
count= 1000

real    0m0.112s
user    0m0.106s
sys     0m0.005s
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ time python wordfreq.py
count= 1000

real    0m0.117s
user    0m0.104s
sys     0m0.008s
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$
```

图 20: 原始词频统计程序的运行时间

2.4.3 分析原始程序主要耗时位置

使用 `python -m cProfile -s cumulative wordfreq.py` 分析耗时分布，发现 `split`, `read` 等函数几乎不耗时，`append` 函数虽然执行 1000 次但总耗时极少，反倒是模块顶层代码耗费了绝大多数的时间。据此分析，应当是因为 `if word not in unique` 这行判断使用了 `in` 操作符查找列表内容，而其时间复杂度高达 $O(n)$ 。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q00$ python -m cProfile -s cumulative wordfreq.py
count= 1000
      1012 function calls in 0.103 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.103   0.103   0.103   0.103 {built-in method builtins.exec}
      1   0.102   0.102   0.103   0.103 wordfreq.py:1(<module>)
      1   0.001   0.001   0.001   0.001 {method 'split' of 'str' objects}
      1   0.000   0.000   0.000   0.000 {method 'read' of '_io.TextIOWrapper' objects}
    1000   0.000   0.000   0.000   0.000 {method 'append' of 'list' objects}
      1   0.000   0.000   0.000   0.000 {built-in method builtins.print}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:319(decode)
      1   0.000   0.000   0.000   0.000 {built-in method _codecs.utf_8_decode}
      1   0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000   0.000   0.000   0.000 {built-in method _io.open}
      1   0.000   0.000   0.000   0.000 {built-in method builtins.len}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:309(__init__)
      1   0.000   0.000   0.000   0.000 <frozen codecs>:260(__init__)
```

图 21: 原始程序耗时分布

2.4.4 优化程序并统计优化前后中位数和加速比

根据之前的分析，我们可以将 `unique` 变成集合，这样可以将查询的时间复杂度降到近乎 $O(1)$ ，同时把 `append` 改为 `add`。优化后程序运行时间降到 0.014s 和 0.015s。优化前后程序运行时间中位数分别为 0.1145s 和 0.0145 s，加速比可达 7.897。

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ vim ./wordfreq.py
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ cat ./wordfreq.py
# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = set() # 使用集合
for word in words: # 集合查找接近 O(1)
    if word not in unique:
        unique.add(word) # 加入集合
print("count=", len(unique))
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ time python wordfreq.py
count= 1000

real    0m0.015s
user    0m0.004s
sys     0m0.009s
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$ time python wordfreq.py
count= 1000

real    0m0.014s
user    0m0.006s
sys     0m0.006s
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q08$

```

图 22: 优化后的程序及其运行时间

3 课后练习

3.1 第 01 题: - 的用途

3.1.1 题目描述

你可能见过像 `cmd -flag --notafalg` 这样的命令。这里的 `-` 是一个特殊参数，它告诉程序后面不要再继续解析选项（flag）了。也就是说，`-` 后面的所有内容都会被当作位置参数（positional argument）。这有什么用？试着运行 `touch - -myfile`，然后在不使用 `-` 的情况下把它删掉。

3.1.2 解题过程

`-` 可以避免将以 `-` 开头的参数视为选项。想要在不使用 `-` 的情况下删除文件名以 `-` 开头的文件可以通过 `rm ./-myfile`。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ touch -- -myfile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ls -l -- -myfile
-rw-rw-r-- 1 mrw mrw 0 Sep  9 14:02 -myfile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ rm ./-myfile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ls -l -- -myfile
ls: 无法访问 '-myfile': 没有那个文件或目录
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$

```

图 23: -myfile 文件的新建与删除

3.2 第 02 题：使用 ls 按特定格式列出文件

3.2.1 题目描述

阅读 man ls，写出一个 ls 命令，让它按下面的方式列出文件：

- 包含所有文件，包括隐藏文件
- 文件大小以易读格式显示（例如 454M，而不是 454279954）
- 按最近修改时间排序
- 输出带颜色

3.2.2 解题过程

通过 man ls 进行查阅，可以发现短选项 -a 可以列出包含隐藏文件的所有文件，-l 可以列出文件详细信息，-h 可按人类易读方式列出文件大小，-t 可按文件修改时间从新到旧列出文件，而长选项 --color 支持带颜色的输出。故可以使用命令 ls -alht --color 实现题目要求。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ls -alht --color
总计 12K
drwxrwxr-x  2 mrw mrw 4.0K Sep  9 14:17 .
-rw-rw-r--  1 mrw mrw 3.0K Sep  8 18:29 lists.txt
drwxrwxr-x 21 mrw mrw 4.0K Sep  8 16:56 ..

```

图 24: 使用 ls -alht --color 后的输出结果

3.3 第 03 题：printenv 与 export 的差异比较

3.3.1 题目描述

进程替换 `<(command)` 可以让你把一个命令的输出当成文件来用。试着配合 `diff` 和进程替换，比较 `printenv` 与 `export` 的输出。它们为什么不一样？（提示：可以试试 `diff <(printenv | sort) <(export | sort)`）

3.3.2 解题过程

`printenv` 和 `export` 都会输出 shell 的环境变量（会传递给子进程的变量），区别在于 `printenv` 仅会输出环境变量键值对，而 `export` 会输出 shell 属性标记如 `declare -x VAR=VALUE`。

```
< TERM=xterm-256color
< USER=mrw
< USERNAME=mrw
< _=/usr/bin/printenv
< VTE_VERSION=7600
< WAYLAND_DISPLAY=wayland-0
< XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.OJ7VU3
< XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
< XDG_CURRENT_DESKTOP=ubuntu:GNOME
< XDG_DATA_DIRS=/usr/share/ubuntu:/usr/share/gnome:/usr/local/share:/usr/share:/var/lib/snapd/desktop
< XDG_MENU_PREFIX=gnome-
< XDG_RUNTIME_DIR=/run/user/1000
< XDG_SESSION_CLASS=user
< XDG_SESSION_DESKTOP=ubuntu
< XDG_SESSION_TYPE=wayland
< XMODIFIERS=@im=ibus
---
> declare -x CLUTTER_DISABLE_MIPMAPPED_TEXT="1"
> declare -x COLORTERM="truecolor"
> declare -x DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/1000/bus"
> declare -x DEBUGINFOD_URLS="https://debuginfod.ubuntu.com "
> declare -x DESKTOP_SESSION="ubuntu"
> declare -x DISPLAY=":0"
> declare -x GDMSESSION="ubuntu"
> declare -x GNOME_DESKTOP_SESSION_ID="this-is-deprecated"
> declare -x GNOME_SETUP_DISPLAY=":1"
> declare -x GNOME_SHELL_SESSION_MODE="ubuntu"
> declare -x GNOME_TERMINAL_SCREEN="/org/gnome/Terminal/screen/bc727ab1_e229_426f_8ccf_ea93ca62350a"
> declare -x GNOME_TERMINAL_SERVICE=":1.103"
> declare -x GSM_SKIP_SSH_AGENT_WORKAROUND="true"
```

图 25: 命令执行后的部分输出

3.4 第 04 题：利用 bash 函数与环境变量保存工作目录

3.4.1 题目描述

写两个 bash 函数 marco 和 polo，行为如下：每次执行 marco 时，都要以某种方式保存当前工作目录；之后无论你切到哪个目录，只要执行 polo，它都应该把你 cd 回执行 marco 时所在的目录。为了方便调试，你可以把代码写进 marco.sh，然后通过执行 source marco.sh 把这些定义重新加载到当前 shell。

3.4.2 解题过程

编写 marco 函数使其执行时将 pwd 命令的输出（PWD 变量的值不和工作目录完全绑定）保存到变量 marco_dir 中，再编写 polo 函数使其执行时通过 cd 命令修改当前目录。据此编写 marco.sh 并通过 source marco.sh 将两个函数加载到 shell 终端里。

Listing 1: marco.sh 的内容

```
1 marco() {  
2     marco_dir=$(pwd)  
3 }  
4 polo() {  
5     cd $marco_dir || echo "No dir saved."  
6 }
```

```
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ vim ./marco.sh  
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ source marco.sh  
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ marco  
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ cd /  
nrw@nrw-VMware-Virtual-Platform:/$ polo  
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ cd ~  
nrw@nrw-VMware-Virtual-Platform:~$ polo  
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week02$
```

图 26: 函数效果演示

3.5 第 05 题：对产生失败不确定的命令进行调试

3.5.1 题目描述

假设你有一个很少失败的命令。为了调试它，你需要把它的输出保存下来，但等到一次失败运行可能会很耗时。写一个 bash 脚本，不断运行下面这个脚本直到它失败为止，并把标准输出和标准错误分别保存到文件里，最后把结果打印出来。如果 > 你还能顺便报告它运行了多少次才失败，就加分。

Listing 2: 给定的脚本内容

```
1  #!/usr/bin/env bash
2
3  n=$(( RANDOM % 100 ))
4
5  if [[ n -eq 42 ]]; then
6      echo "Something went wrong"
7      >&2 echo "The error was using magic numbers"
8      exit 1
9  fi
10
11 echo "Everything went according to plan"
```

3.5.2 解题过程

将给定内容存入 42.sh 脚本，并赋予其可执行权限。随后新建 test 脚本，定义变量 cnt=0 用以记录 42.sh 成功执行次数，将 ./42.sh » stdout.log 2» stderr.log 作为 while 语句的执行条件，若语句成功则 while 循环继续执行使计数加 1，若执行失败则跳出 while 循环并输出成功执行的次数。

Listing 3: test.sh 脚本内容

```
1  #!/bin/bash
2  cnt=0
3  while ./42.sh >> stdout.log 2>> stderr.log
4  do
```

```
5 cnt=$((cnt+1))
6 done
7
8 echo "Failed after $cnt runs."
```

```
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ vim ./42.sh
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ chmod +x ./42.sh
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ vim ./test.sh
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ chmod +x ./test.sh
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ./test.sh
Failed after 29 runs.
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ cat sy dout.log
cat: sy dout.log: 没有那个文件或目录
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ cat stdout.log
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Everything went according to plan
Something went wrong
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ cat stderr.log
The error was using magic numbers
nrrw@nrrw-VMware-Virtual-Platform:~/sys-dev-test/week02$
```

图 27: test.sh 的执行效果

3.6 第 06 题：任务管理训练

3.6.1 题目描述

在终端里启动一个 `sleep 10000` 任务，用 `Ctrl-Z` 把它挂起，再用 `bg` 让它继续在后台运行。然后使用 `pgrep` 找到它的 `pid`，再用 `pkill` 把它杀掉，整个过程都不要手动输入这个 `pid`。（提示：试试 `-af` 参数）

3.6.2 解题过程

利用 `pgrep -af` 选项可以根据进程名称查询并展示查找到的 `PID` 和命令行，利用 `pkill -f` 可以通过进程名称匹配要结束的进程。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ sleep 10000
^Z
[1]+  已停止                  sleep 10000
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ bg
[1]+  sleep 10000 &
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ pgrep -af 'sleep 10000'
4576 sleep 10000
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ pkill -f 'sleep 10000'
[1]+  已终止                  sleep 10000
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ pgrep -af 'sleep 10000'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$
```

图 28: 进程控制演示结果

3.7 第 07 题：利用 `alias` 命令创建 `cd` 命令的别名

3.7.1 题目描述

创建一个 `alias dc`，让你把 `cd` 打错时也能正常工作。

3.7.2 解题过程

使用命令 `alias dc="cd"` 为 `cd` 命令创建别名 `dc` 即可。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ alias dc="cd"
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ dc ./
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ dc ../
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ cd ./week02
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$
```

图 29: cd 别名 dc 的创建与使用效果

3.8 第 08 题：配置 ssh 密钥

3.8.1 题目描述

进入 `./ssh/`，检查你是否已经有一对 SSH 密钥。如果没有，就用 `ssh-keygen -a 100 -t ed25519` 生成一对。建议给密钥设置密码，并配合 `ssh-agent` 使用。

3.8.2 解题过程

执行 `ssh-keygen -a 100 -t ed25519 -f ./ssh/id_ed25519` 生成 ssh 密钥并输入使用密钥所需的密码，随后使用 `eval "$(ssh-agent -s)"` 启动 ssh 代理进程并向终端注入环境变量，最后用 `ssh-add ./ssh/id_ed25519` 将生成的 ssh 密钥文件加载到代理中，使得代理运行期间无需再次输入密码验证 ssh 密钥。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ssh-keygen -a 100 -t ed25519 -f ~/.ssh/id_ed25519
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/mrw/.ssh/id_ed25519
Your public key has been saved in /home/mrw/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:WITL26xeRCiNGG1qYALDcQwZCOAyTxrwJet6I68xBw4 mrw@mrw-VMware-Virtual-Platform
The key's randomart image is:
+--[ED25519 256]--+
| %+0o.  ..      |
| **.Boo.o       |
| *o=oo.o.o.o    |
|.Bo .o+        |
|E.o .+S        |
|oo  ..o        |
|=.+  ..        |
| B .  ..        |
|... ..         |
+----[SHA256]-----+
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ eval "$(ssh-agent -k)"
SSH_AGENT_PID not set, cannot kill agent
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ eval "$(ssh-agent -s)"
Agent pid 4947
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/mrw/.ssh/id_ed25519:
Identity added: /home/mrw/.ssh/id_ed25519 (mrw@mrw-VMware-Virtual-Platform)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$

```

图 30: ssh 密钥的生成与 ssh-agent 的配置

3.9 第 09 题：编辑 ssh 配置

3.9.1 题目描述

编辑`ssh/config`，加入下面这样的配置项：

Listing 4: 给定的配置项

```

1 Host vm
2     User username_goes_here
3     HostName ip_goes_here
4     IdentityFile ~/.ssh/id_ed25519
5     LocalForward 9999 localhost:8888

```

3.9.2 解题过程

通过 ifconfig 获取另一台虚拟机的本地 ip 为 192.168.234.129。用户名为 kali，据此用 vim 填写配置并进行保存。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ vim ~/.ssh/config
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ cat ~/.ssh/config
Host vm
  User kali
  HostName 192.168.234.129
  IdentityFile ~/.ssh/id_ed25519
  LocalForward 9999 localhost:8888
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$
```

图 31: 编写的 ssh 配置内容

3.10 第 10 题：复制 ssh 公钥到服务器上

3.10.1 题目描述

使用 ssh-copy-id vm 把你的 SSH 公钥复制到服务器上。

3.10.2 解题过程

运行 ssh-copy-id vm 尝试复制 ssh 公钥到服务器上，由于这是第一次连接程序表示主机名称未知，向我们请求是否信任主机，输入 yes 确认连接。随后程序提示我们输入 kali 用户的密码，输入其密码后 ssh 密钥被添加到虚拟机 kali 用户的 authorized_keys 中。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$ ssh-copy-id vm
The authenticity of host '192.168.234.129 (192.168.234.129)' can't be established.
ED25519 key fingerprint is SHA256:zht4uUudfbye4AW+ZkW3Cstx5fdBd1mE2kkLBY6UkU0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kali@192.168.234.129's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'vm'"
and check to make sure that only the key(s) you wanted were added.

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week02$
```

图 32: 使用 ssh-copy-id 将 ssh 密钥复制到虚拟机账户

3.11 第 11 题: 启动 Web 服务器并尝试访问

3.11.1 题目描述

在虚拟机里执行 `python -m http.server 8888` 启动一个 Web 服务器。然后在你自己的机器上访问 `http://localhost:9999`，确认自己能访问虚拟机里的这个服务。

3.11.2 解题过程

在虚拟机的 kali 账户上执行 `python -m http.server 8888` 在虚拟机本地的 8888 端口启动 python 服务器，随后使用 `ssh vm` 远程登录到虚拟机的 kali 账户。由于 vm 配置中配置了前向转发，在本地的 9999 端口对应到虚拟机的 8888 端口，此时在本地访问 `http://localhost:9999` 即可访问到虚拟机上的 python 服务器。

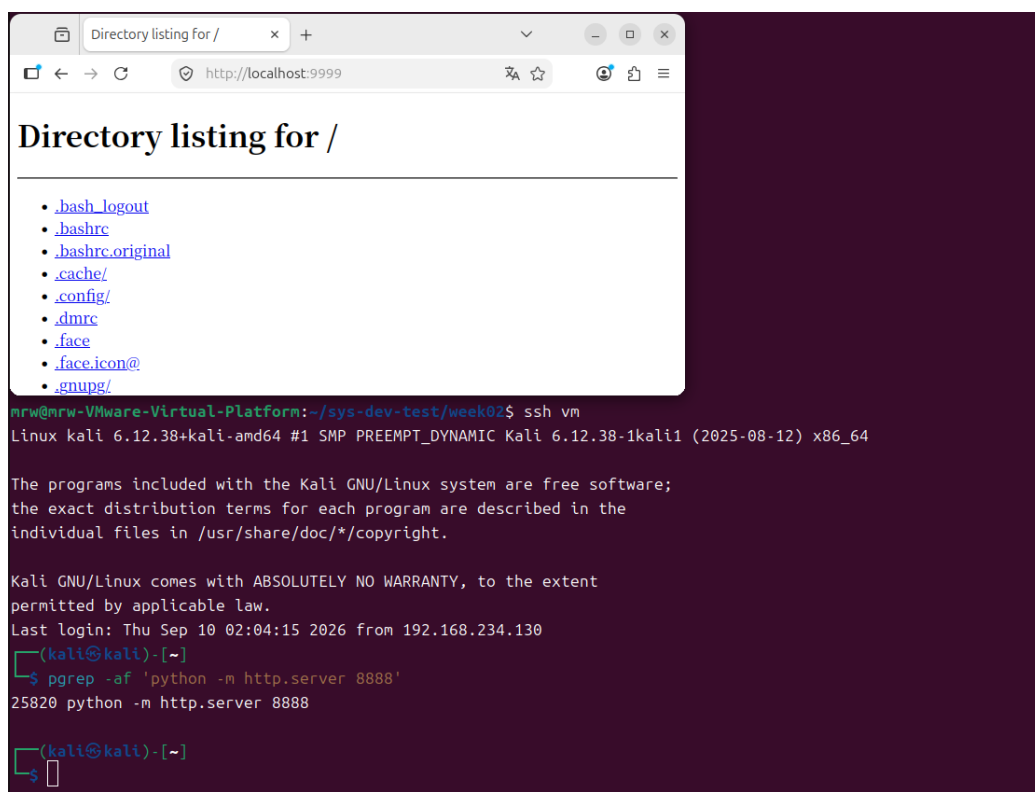


图 33: 利用 ssh 连接到虚拟机并访问 python 服务器

4 commit 记录

本周实验所使用的文件的上传记录如下，详细信息请访问 github 仓库。

